SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem Solving Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Assessment Tool Weightage: 20%

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 40**
Question Count: 5

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Industry Problem Solving Task

1. An e-commerce company receives clickstream, transaction, and review data from millions of users daily. Propose a big data analytics pipeline and explain how the 5Vs influence architecture design.
2. A healthcare organization collects structured patient records, semi-structured insurance claims, and unstructured medical images. Design an approach to manage and secure this data ecosystem.
3. A telecom company wants to detect churn patterns using high-volume customer interaction data. Identify the big data challenges and recommend suitable analytics and integration strategies.
4. A smart city project generates sensor streams from traffic, air quality, and surveillance systems. Explain how distributed file systems and data integration tools can support large-scale analysis.
5. A financial analytics firm must comply with strict privacy requirements while processing high-velocity transaction data. Design a secure big data solution and justify the controls chosen.

Industry Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Context	20%	Demonstrates strong understanding of real-world problem context	Good understanding	Basic understanding	Weak understanding
Application of Big Data Concepts	25%	Applies highly relevant big data ideas and tools	Mostly appropriate application	Partial application	Weak application

Solution Design	25%	Solution is practical, scalable, and well reasoned	Reasonable solution	Basic solution with gaps	Unclear solution
Security / Risk Awareness	15%	Strong awareness of security and operational risks	Reasonable awareness	Limited awareness	Poor awareness
Clarity and Professionalism	15%	Well-structured and professional response	Generally clear	Moderately clear	Poorly presented

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem Solving Task — Questions with Answers

COURSE NAME:	Cloud Computing and Big Data Analytics	**COURSE CODE:**	CSA15
Assessment Tool:	Tool 2 – Industry Problem Solving (Weightage 20%)	**Total Marks:**	40
Taxonomy:	Dave's Taxonomy – Articulation (Level 4)	**Question Count:**	5

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks — Industry Problem Solving (Answers)

Question 1

An e-commerce company receives clickstream, transaction, and review data from millions of users daily. Propose a big data analytics pipeline and explain how the 5Vs influence architecture design.

Answer

1. Problem understanding

- Three very different feeds: **clickstream** (huge volume, high velocity, semi-structured JSON), **transactions** (structured, ACID-critical, lower volume) and **reviews** (unstructured free text plus images).
- Business goals: real-time personalisation and recommendations, fraud/abuse detection, inventory and demand forecasting, and sentiment/quality monitoring.

2. Proposed pipeline — layer by layer

Sources → Ingestion (Kafka / Sqoop / CDC) → Raw Landing Zone (S3 / HDFS) → Processing (Spark batch + Flink/Spark Streaming) → Curated Store (Parquet lakehouse + Hive/Snowflake + Cassandra serving layer) → Analytics & ML (feature store, models) → Consumption (site personalisation API, BI dashboards)

Layer	Components / Tools	Design Justification
Data sources	Web & mobile SDK events, order microservice DB, review service, third-party ad and logistics APIs	Clicks are fire-and-forget events; orders are transactional and must not be lost
Ingestion	Apache Kafka (partitioned by user_id) for clicks and reviews; Debezium CDC / Sqoop for the order database; NiFi for API pulls	Kafka decouples producers from consumers, buffers traffic spikes (festival sales) and replays on failure; CDC avoids nightly full dumps

Raw storage	Amazon S3 / HDFS data lake, partitioned by *event_date/hour/event_type*; Avro on ingest, Parquet after compaction	Schema-on-read keeps all raw fidelity; columnar Parquet cuts scan cost 5–10× for analytics
Stream processing	Spark Structured Streaming / Apache Flink; Redis for session state	Sessionisation, bot filtering, real-time "customers also viewed" counters, cart-abandonment triggers — latency in seconds
Batch processing	Apache Spark on YARN/Kubernetes, orchestrated by Airflow	Nightly deduplication, joins across clicks + orders + reviews, aggregate tables, model retraining
Curated / serving	Hive/Delta Lake tables for analysts; Cassandra or DynamoDB for low-latency lookups; Elasticsearch for review search	Different query patterns need different stores — polyglot persistence
Analytics & ML	Collaborative filtering + gradient-boosted ranking, NLP sentiment on reviews (BERT), demand forecasting; feature store	Recommendations drive the primary revenue uplift
Consumption	Personalisation API (<100 ms), Tableau/Superset dashboards, A/B testing framework	Value is realised only when the model output reaches the storefront
Governance	Apache Atlas lineage, Ranger access control, PII tokenisation, TLS + at-rest encryption	PCI-DSS for card data, privacy law for user profiles

3. How each V influences the architecture

V	Manifestation in this Case	Architectural Decision Forced
Volume	Billions of click events, several TB/day, years of history retained	Distributed object storage instead of an RDBMS; date partitioning; Parquet + Snappy compression; lifecycle tiering of cold data to Glacier; horizontal scale-out clusters
Velocity	Peak of 100k+ events/sec during sale events; recommendations must reflect the last few clicks	Kafka buffer with many partitions; a **Lambda / Kappa hybrid** — a speed layer for seconds-fresh signals and a batch layer for accurate history; auto-scaling consumers
Variety	Structured orders, semi-structured JSON clicks, unstructured review text and photos	Data lake with schema-on-read; separate NoSQL, columnar and search stores (polyglot persistence); schema registry to manage event evolution
Veracity	Bot traffic, duplicate events from retries, fake reviews, missing IDs across devices	Deduplication on event UUID, bot-filtering rules, identity resolution/stitching, data-quality checks in Airflow, review-fraud classifier
Value	Uplift in conversion, basket size and retention must exceed platform cost	Build for the highest-ROI use case first (recommendations); measure via A/B tests; keep only data with a defined consumer to control cost

> **Justification of key trade-off:** A pure batch design would be cheaper but cannot personalise within a session; a pure streaming design is expensive and hard to backfill. The **hybrid speed + batch layer** gives sub-second personalisation while retaining a re-computable, accurate historical record.

Question 2

A healthcare organization collects structured patient records, semi-structured insurance claims, and unstructured medical images. Design an approach to manage and secure this data ecosystem.

Answer

1. Data landscape

Data Category	Type	Volume / Velocity	Recommended Store & Format
Patient records (EHR)	Structured	Moderate volume, continuous updates	Relational / columnar store (PostgreSQL → Delta Lake), HL7 FHIR standard
Insurance claims	Semi-structured (XML/JSON, X12)	Batch, high variability of fields	Document store (MongoDB) or Parquet with nested schema
Medical images (MRI/CT/X-ray)	Unstructured, DICOM, very large objects	Very high volume (GB per study)	Object storage (S3/HDFS) with a PACS/DICOM index; metadata in a catalogue
Device / ICU telemetry	Streaming time-series	Continuous, high velocity	Kafka → time-series DB (InfluxDB / Timescale)
Clinical notes	Unstructured text	Moderate	Data lake + Elasticsearch; NLP pipeline for entity extraction

2. Management architecture

Ingest (HL7/FHIR gateway, DICOM router, claims SFTP, Kafka for device streams) → **Bronze raw zone** (immutable, encrypted) → **Silver cleansed zone** (validated, standardised, de-identified copies) → **Gold curated zone** (analytics marts, feature store) → **Consumption** (clinical dashboards, research sandbox, ML models)

- **Medallion (bronze/silver/gold) lake design** keeps the raw legal record untouched while allowing cleansing and de-identification downstream.
- **Common patient identifier / MDM** resolves the same patient across EHR, claims and imaging systems — the single most important integration step.
- **Metadata catalogue (Apache Atlas)** stores image pointers and clinical context so a 500 MB DICOM study need not be scanned to be found.
- **Interoperability standards** — HL7 FHIR for records, X12 for claims, DICOM for imaging — avoid custom point-to-point integrations.
- **Tiered retention:** hot storage for recent studies, cold/archive tier for older ones to control cost under legal retention rules.

3. Security and privacy design (defence in depth)

Control Layer	Mechanism	Purpose / Justification
Identity	Kerberos / SSO with MFA, service accounts for jobs	Establishes *who* before any data access; no anonymous cluster access
Authorisation	RBAC + ABAC via Apache Ranger; column- and row-level policies	A billing clerk sees claim amounts but not diagnoses; least privilege enforced
Encryption in transit	TLS 1.3 on every hop, including intra-cluster RPC and shuffle	Prevents interception on the internal network
Encryption at rest	HDFS/S3 encryption zones, envelope encryption with KMS, key rotation	A stolen disk or snapshot yields unreadable data

De-identification	Tokenisation/pseudonymisation of MRN, name, address; DICOM header scrubbing; k-anonymity for research extracts	Researchers can analyse without ever touching direct identifiers
Data masking	Dynamic masking in query layer based on user role	One physical dataset serves many trust levels
Network	Private subnets, no public endpoints, bastion access, segmented clinical VLAN	Shrinks the attack surface of a distributed cluster
Audit & monitoring	Immutable audit logs to SIEM, access anomaly detection, lineage in Atlas	Proves compliance and catches insider misuse
Consent & governance	Consent registry, purpose-of-use tagging, data stewardship committee	Legal basis for each processing purpose is recorded and enforceable
Resilience	Cross-region replication, tested restore, ransomware-resistant immutable backups	Availability is a patient-safety issue, not just an IT metric

4. Compliance mapping

- **HIPAA / GDPR / DPDP Act:** satisfied by encryption, minimum-necessary access, audit trails, breach notification process and the right to erasure (supported by tokenised identifiers, so deleting the token map effectively erases the subject).
- **Retention:** policy-driven lifecycle rules per data class rather than "keep everything forever".

> **Justification of the controls chosen:** Healthcare's dominant risk is not scale but **confidentiality and integrity**. Every control above is selected to make identifiable data unreadable, unreachable, or traceable — while still letting analytics run on de-identified copies, so security does not block clinical value.

> **Question 3**
>
> A telecom company wants to detect churn patterns using high-volume customer interaction data. Identify the big data challenges and recommend suitable analytics and integration strategies.

Answer

1. Data involved

- **CDRs** (call detail records) — billions of rows per day, structured, extremely high volume.
- **Network / data-session logs** — dropped calls, latency, tower congestion (the strongest churn predictors).
- **Customer care** — ticket history, IVR paths, call-centre voice recordings (unstructured).
- **Billing & plans** — structured, plus recharge and payment history.
- **App usage, roaming, device changes, and social/complaint sentiment.**

2. Big data challenges identified

Challenge	Description in Telecom Context	Mitigation
Volume	Billions of CDRs daily; multi-PB history needed to see long-term behaviour trends	Distributed lake, date+circle partitioning, Parquet/ORC compression, aggregate rollups instead of raw scans
Velocity	Churn signals (repeated dropped calls, competitor SIM ported) must trigger a retention offer within hours, not next month	Kafka streams + Flink for real-time trigger events feeding a campaign engine
Variety & silos	CDR, CRM, billing, network and care systems are separate legacy platforms with different customer keys	MDM / identity resolution on MSISDN + account ID; a unified 360° customer profile table

Veracity	Duplicate CDRs from switch retries, missing tower IDs, mis-tagged tickets, multiple SIMs per person	Deduplication keys, validation rules at ingestion, household/person linkage
Class imbalance	Monthly churn may be only 2–3% of the base — naive models predict "no churn" and score 97% accuracy while being useless	SMOTE / class weighting; evaluate on **precision-recall, AUC and lift**, never plain accuracy
Concept drift	A competitor's new tariff changes churn behaviour overnight, degrading the model silently	Continuous monitoring, scheduled retraining, champion/challenger models
Privacy & regulation	CDRs and location data are highly sensitive and regulated by the telecom authority	Masking of MSISDN, restricted access, purpose limitation, audit logging
Actionability	A prediction with no linked retention offer generates zero value	Close the loop into the campaign management / CRM system and measure saved-customer rate

3. Recommended integration strategy

CDR mediation & network probes → Kafka → HDFS/S3 lake; **CRM & billing DBs → Sqoop / Debezium CDC →** lake; **call recordings →** object store **→ speech-to-text + NLP → Spark feature engineering → Customer 360 table + feature store → churn model → retention campaign engine**

- **Airflow** orchestrates daily feature builds and weekly retraining, with SLA alerts.
- **Hive/Impala or Presto** gives analysts SQL access without moving data.
- **Feature store** guarantees the same feature definitions are used in training and in real-time scoring (prevents training/serving skew).

4. Recommended analytics strategy

Analytics Type	Technique	Churn Insight Delivered
Descriptive	Cohort and segment churn dashboards	Which plans, circles and tenures leak customers
Diagnostic	Correlation of churn with dropped-call rate, complaint count, billing disputes	Root cause — e.g. churn concentrated around three congested towers
Predictive	Gradient boosting (XGBoost/LightGBM), survival analysis for time-to-churn, RFM features	Ranked list of at-risk customers with a churn probability
Prescriptive	Uplift modelling + offer optimisation	Which offer to whom — targets only customers whose behaviour the offer will actually change
Graph / social	Call-graph community detection	Influencer churn cascades — when one hub leaves, their circle follows
Text / voice	NLP sentiment on care transcripts and social posts	Early dissatisfaction signals invisible in structured data

> **Key recommendation:** Use **uplift modelling rather than plain churn probability**. Many high-probability churners will leave regardless, and discounting them wastes margin; targeting the *persuadable* segment maximises retained revenue per rupee of incentive.

Question 4

A smart city project generates sensor streams from traffic, air quality, and surveillance systems. Explain how distributed file systems and data integration tools can support large-scale analysis.

Answer

1. Nature of smart-city data

Source	Data Type	Rate / Size	Latency Requirement
Traffic loop sensors, ANPR cameras, GPS from buses	Structured time-series + images	Thousands of readings/sec	Seconds — signal control, congestion alerts
Air quality monitors (PM2.5, NO_2, CO)	Structured time-series	Low rate, many endpoints	Minutes — public advisories
Surveillance CCTV	Unstructured video	Very high — TB per day per zone	Real-time for incidents; long retention for forensics
Weather, events, utility, citizen apps	Semi-structured APIs	Periodic	Hours — contextual enrichment

2. Role of the distributed file system

- **Scale-out capacity:** HDFS or cloud object storage absorbs petabytes of video and years of sensor history that no single server or SAN could hold, and grows by adding nodes.
- **Fault tolerance:** 3× block replication (or erasure coding for cold video, cutting overhead from 200% to ~50%) means a failed disk never loses evidence.
- **Data locality & parallel throughput:** Spark jobs analysing a month of traffic data run on the nodes holding the blocks, so hundreds of disks stream in parallel instead of one network link becoming the bottleneck.
- **Format agnosticism:** the same lake holds Parquet sensor tables, JSON API dumps and raw MP4 video — enabling cross-domain correlation (e.g. traffic density vs. PM2.5).
- **Partitioning strategy:** organise by *city_zone / date / hour / sensor_type* so a query touches a few directories instead of the whole lake; compact small IoT files into large Parquet files to avoid the HDFS small-file problem.
- **Tiered lifecycle:** hot SSD tier for the last 7 days, warm HDD for a quarter, archive tier for retention-mandated video — keeping cost sustainable for a public budget.

3. Role of integration tools

Tool	Function in the Smart City Pipeline	Why It Is Needed
MQTT broker / IoT gateway	Lightweight protocol for constrained, intermittently connected field sensors	HTTP is too heavy for battery/low-bandwidth devices
Apache NiFi	Edge and gateway routing, protocol conversion, filtering, provenance tracking	Handles hundreds of heterogeneous device protocols with visual, low-code flows
Apache Kafka	Central event backbone, topic per sensor domain, buffering and replay	Decouples thousands of producers from many consumers; absorbs bursts (rush hour, festivals)
Flume / Fluentd	Log and file collection from edge servers into HDFS	Reliable, buffered delivery of bulk files
Spark Streaming / Flink	Windowed aggregation, geospatial joins, anomaly detection in flight	Congestion and pollution-spike alerts must fire during the event, not after
Sqoop / JDBC connectors	Pull municipal records — vehicle registration, ward boundaries, permits	Enriches sensor data with authoritative reference data
Apache Airflow	Schedules daily aggregation, model retraining, report generation	Dependency management, retries and SLA monitoring across many pipelines
Apache Atlas / Ranger	Lineage, cataloguing and access control	Surveillance data demands strict, auditable access — a civil-liberties requirement

4. Reference architecture (Lambda pattern)

Sensors / CCTV → Edge nodes (filter, compress, run light inference so only events, not all frames, travel) → **Kafka** → split into **Speed layer** (Flink → real-time dashboards, signal control, incident alerts) and **Batch layer** (HDFS/S3 → Spark → historical models, city planning) → **Serving layer** (time-series DB + GIS dashboards for city operations centre)

- **Edge/fog pre-processing is essential** — sending every CCTV frame to the centre would saturate the city network; sending only detected events cuts bandwidth by orders of magnitude.
- **Geospatial indexing** (geohash / H3 cells) makes "all sensors within 500 m of this junction" a cheap lookup.

> **Conclusion:** The DFS provides the **scalable, fault-tolerant, format-neutral memory of the city**, while integration tools provide its **nervous system**, moving and harmonising signals from thousands of dissimilar devices. Neither alone is sufficient: storage without integration yields disconnected silos; integration without scalable storage has nowhere to land.

Question 5

A financial analytics firm must comply with strict privacy requirements while processing high-velocity transaction data. Design a secure big data solution and justify the controls chosen.

Answer

1. Requirements analysis

Requirement	Detail	Design Consequence
High velocity	Tens of thousands of transactions/second; scoring within ~200 ms	Streaming architecture; security controls must be low-overhead
Strict privacy	PCI-DSS for card data, GDPR/DPDP for personal data, SOX/RBI reporting	Encryption, tokenisation, data residency, auditability are non-negotiable
High integrity	Financial records must be tamper-evident and reconcilable	Immutable append-only storage, checksums, WORM archives
High availability	Downtime blocks payments and breaches SLAs	Multi-AZ replication, tested DR, graceful degradation

2. Secure architecture

Transaction sources → TLS-protected API gateway → Tokenisation service (PAN → token at the edge) → Kafka (encrypted, mTLS, ACL per topic) → Flink/Spark Streaming scoring + rules engine → Encrypted lake (S3/HDFS encryption zones) + low-latency store (Redis/Cassandra) → Analytics & ML in a restricted enclave → Masked dashboards / alert queue; all layers feeding an **immutable audit log → SIEM**

3. Controls and their justification

#	Control	Implementation	Justification
1	**Tokenisation at the edge**	Card PAN replaced with a surrogate token immediately on entry; vault holds the mapping in a separate, hardened zone	The analytics platform never stores real card numbers, so it falls largely **out of PCI-DSS scope** — the single highest-leverage control
2	**Encryption in transit**	TLS 1.3 externally; mutual TLS between Kafka brokers, producers and consumers	High-velocity data crosses many hops; interception anywhere would expose full transaction streams

3	**Encryption at rest + KMS**	HDFS/S3 encryption zones, envelope encryption, HSM-backed keys, automatic rotation, separation of key custodians from data admins	Protects backups, snapshots and decommissioned disks; key separation stops an admin from unilaterally reading data
4	**Strong authN / authZ**	Kerberos or OIDC + MFA; Apache Ranger RBAC/ABAC with column- and row-level policies; service accounts per job	Enforces least privilege — a fraud analyst sees masked PAN and amounts, not customer identity
5	**Dynamic masking & differential privacy**	Masked views by role; noise added to aggregate research extracts	Allows analytics and model building without exposing individual-level personal data
6	**Network segmentation**	Private VPC subnets, no public endpoints, bastion/jump host, micro-segmentation between tiers, WAF at the edge	Contains lateral movement; a compromised consumer node cannot reach the token vault
7	**Immutable audit logging**	Every access and admin action logged to WORM storage, streamed to SIEM, with UEBA anomaly detection	Required evidence for regulators; detects insider misuse and slow exfiltration
8	**Data lineage & catalogue**	Apache Atlas; PII tagging drives policy automatically	Supports GDPR subject-access and erasure requests, and proves where regulated data flows
9	**Retention & residency policy**	Automated lifecycle deletion; region-pinned storage; cross-border transfer controls	Data-localisation rules and the principle of storage limitation
10	**Secure SDLC & ops**	Secrets manager (no credentials in code), image scanning, IaC with policy-as-code, periodic penetration testing	Most breaches stem from misconfiguration, not broken cryptography
11	**Resilience & DR**	Multi-AZ Kafka replication, tested restores, immutable ransomware-resistant backups	Availability and integrity are regulatory obligations alongside confidentiality
12	**Model governance**	Explainable models for adverse decisions, bias testing, versioned model registry	Regulators require that a declined or flagged customer can be given a reason

4. Balancing security against velocity

- Heavy controls are applied **once at the edge** (tokenisation, TLS termination) rather than repeatedly inside the hot path.
- Hardware-accelerated AES keeps at-rest encryption overhead in low single-digit percentages.
- Authorisation decisions are cached per session instead of being evaluated per record.
- The scoring path reads pre-computed features from an in-memory store, so no expensive joins occur under latency pressure.

> **Justification summary:** The design follows **defence in depth** and **privacy by design**: reduce the sensitive footprint first (tokenise), protect what remains (encrypt, segment), control who touches it (authN/authZ, masking), and prove it continuously (audit, lineage). Because the most costly control — taking the platform out of PCI scope — is applied at ingest, the remaining pipeline stays fast enough for real-time scoring.

— End of Assessment Tool 2 (Industry Problem Solving Task) —